Rapport de Projet :

Système de Gestion des Étudiants

Réalisé par:

Aya ER-Rouidi / Aya Janmajdoub / Roukaya Louk

1.Introduction

Dans le cadre du module Java EE, nous avons réalisé une application web dynamique permettant la gestion des étudiants d'un département universitaire.

L'objectif est de développer une application basée sur les technologies Servlet, JSP, Maven et DAO, en respectant l'architecture MVC (Model – View – Controller).

Cette application permet :

- La gestion des étudiants (CRUD)
- L'authentification des utilisateurs
- La séparation claire entre les différentes couches de l'application

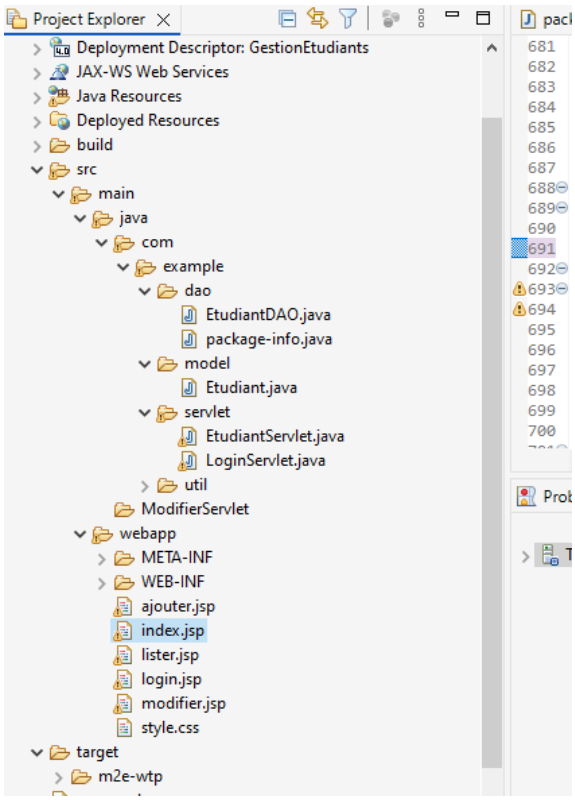
2.Objectifs du projet

- Comprendre le fonctionnement des Servlets
- Manipuler les JSP pour l'affichage dynamique
- Implémenter l'architecture MVC
- Utiliser le modèle DAO pour accéder aux données
- Gérer l'authentification avec session
- Organiser un projet Maven

3. Technologies utilisées

- **Langage** : Java (javax)
- **Framework Web** : Servlets & JSP
- **Gestion de projet** : Maven
- **Base de données** : MySQL
- **Serveur d'application** : Apache Tomcat
- **Front-end** : HTML, CSS
- **Outil de développement** : Eclipse IDE
- **Outil BD** : phpMyAdmin

4. Architecture de l'application



L'application est basée sur l'architecture **MVC (Model – View – Controller)**, largement utilisée dans les applications web Java.

- **Model** : représente les données et la logique métier (classes Java et DAO).
- **View** : représente l'interface utilisateur (pages JSP et HTML).
- **Controller** : assure la communication entre la vue et le modèle (Servlets).

Cette architecture permet une bonne organisation du code, une meilleure maintenance et une séparation claire des responsabilités.

5. Base de données

La base de données utilisée est **MySQL** et contient deux tables principales :

5.1 Table users

Cette table est utilisée pour l'authentification de l'administrateur.

Champs principaux :

- id
- username
- email
- password

5.2 Table etudiant

Cette table contient les informations des étudiants.

Champs principaux :

- id
- cin
- nom
- prénom
- email

6. Description détaillée des Servlets

6.1 LoginServlet

Le LoginServlet est responsable de l'authentification de l'administrateur.

Rôle principal :

- Vérifier les informations de connexion (username et password).
- Créer une session pour l'administrateur.
- Rediriger vers l'espace d'administration.

Méthodes utilisées :

- doPost() :
  - Récupère les paramètres du formulaire de connexion.
  - Vérifie les données dans la table users.
  - Crée une session avec HttpSession.
  - Redirige vers la page principale si l'authentification est réussie.

6.2 EtudiantServlet

Le EtudiantServlet est le contrôleur principal de la gestion des étudiants.

Rôle principal :

- Gérer toutes les opérations CRUD sur les étudiants.

Méthodes utilisées :

- doGet() :
  - Afficher la liste des étudiants.
  - Supprimer un étudiant.
  - Rechercher un étudiant par nom ou par CNE.
- doPost() :
  - Ajouter un nouvel étudiant.
  - Modifier les informations d'un étudiant existant.

Chaque action est déterminée à l'aide de paramètres envoyés depuis les pages JSP.

7. Description des classes DAO

7.1 EtudiantDAO

La classe EtudiantDAO assure la communication entre l'application et la base de données.

Principales méthodes :

- addEtudiant() : insère un nouvel étudiant dans la base de données.
- updateEtudiant() : met à jour les informations d'un étudiant.
- deleteEtudiant() : supprime un étudiant.
- getAllEtudiants() : récupère la liste de tous les étudiants.
- searchEtudiant() : recherche un étudiant par nom ou CNE.

Cette classe utilise PreparedStatement pour éviter les injections SQL.

8. Pages JSP (Vue)

Les pages JSP sont responsables de l'affichage des données et de l'interaction avec l'utilisateur.

Principales pages JSP :

- login.jsp : formulaire de connexion de l'administrateur.
- ajouter.jsp : formulaire d'ajout d'un étudiant.
- modifier.jsp : formulaire de modification d'un étudiant.
- lister.jsp : affichage de la liste des étudiants.
- index.jsp : page d'accueil

Les JSP utilisent les **Expression Language (EL)** pour afficher les données dynamiques.

9. Fonctionnalités de l'application

9-1 Espace Administrateur

L'application dispose uniquement d'un **espace Administrateur**. Cet espace permet la gestion complète des étudiants du département.

Les fonctionnalités principales sont :

- L'authentification de l'administrateur.
- L'ajout d'un étudiant.
- La modification des informations d'un étudiant.
- La suppression d'un étudiant.
- L'affichage de la liste des étudiants.
- La recherche d'un étudiant par nom ou par CNE.

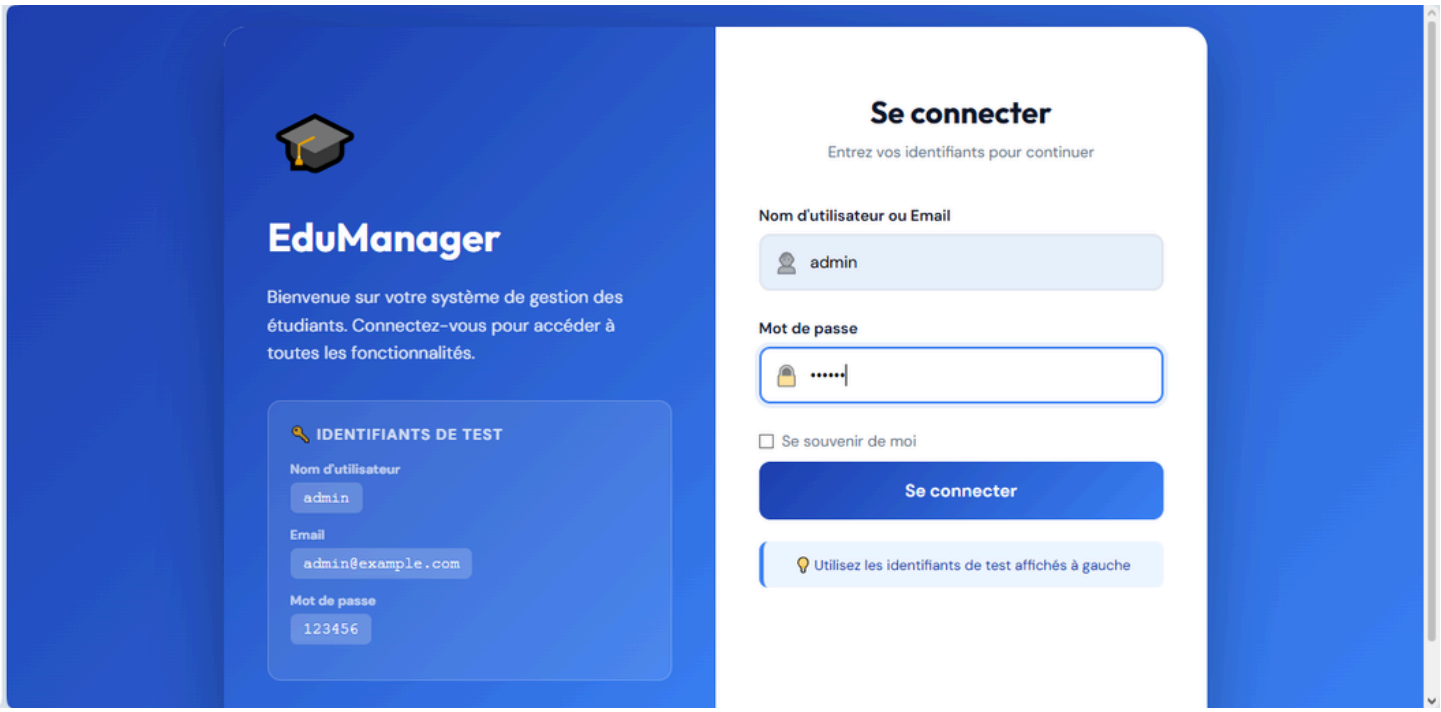
Toutes les opérations de gestion sont centralisées dans cet espace afin de garantir un contrôle total et une meilleure organisation des données.

10. Captures d’écran du projet

Cette section présente des captures d’écran illustrant les principales fonctionnalités de l’application.

10.1 Page de Login

Description : Cette interface permet à l’administrateur de saisir son nom d’utilisateur et son mot de passe afin d’accéder au système. Les données sont envoyées au LoginServlet via la méthode POST.



10.2 page d’accueil

La première image présente la page d'accueil de l'application EduManager. Elle affiche un en-tête de navigation contenant le logo de l'application, ainsi que les liens Accueil, Étudiants, Ajouter et un bouton Déconnexion. Au centre de la page, un titre principal met en avant le slogan « Gérez vos étudiants en toute simplicité », accompagné d'une courte description présentant l'application comme une solution complète et intuitive. Deux boutons d'action sont visibles : Voir les étudiants et Ajouter un étudiant, permettant à l'utilisateur d'accéder rapidement aux fonctionnalités principales.

Image 2 — Section Fonctionnalités

Elle présente trois cartes de fonctionnalités mettant en valeur les points forts de l'application : Ultra rapide, qui souligne la fluidité de la navigation et la rapidité des chargements ; Recherche intelligente, qui permet de retrouver n'importe quel étudiant en quelques secondes grâce à un moteur de recherche avancé ; et 100% Responsive, garantissant un accès optimal depuis tout type d'appareil, qu'il s'agisse d'un ordinateur, d'une tablette ou d'un smartphone.

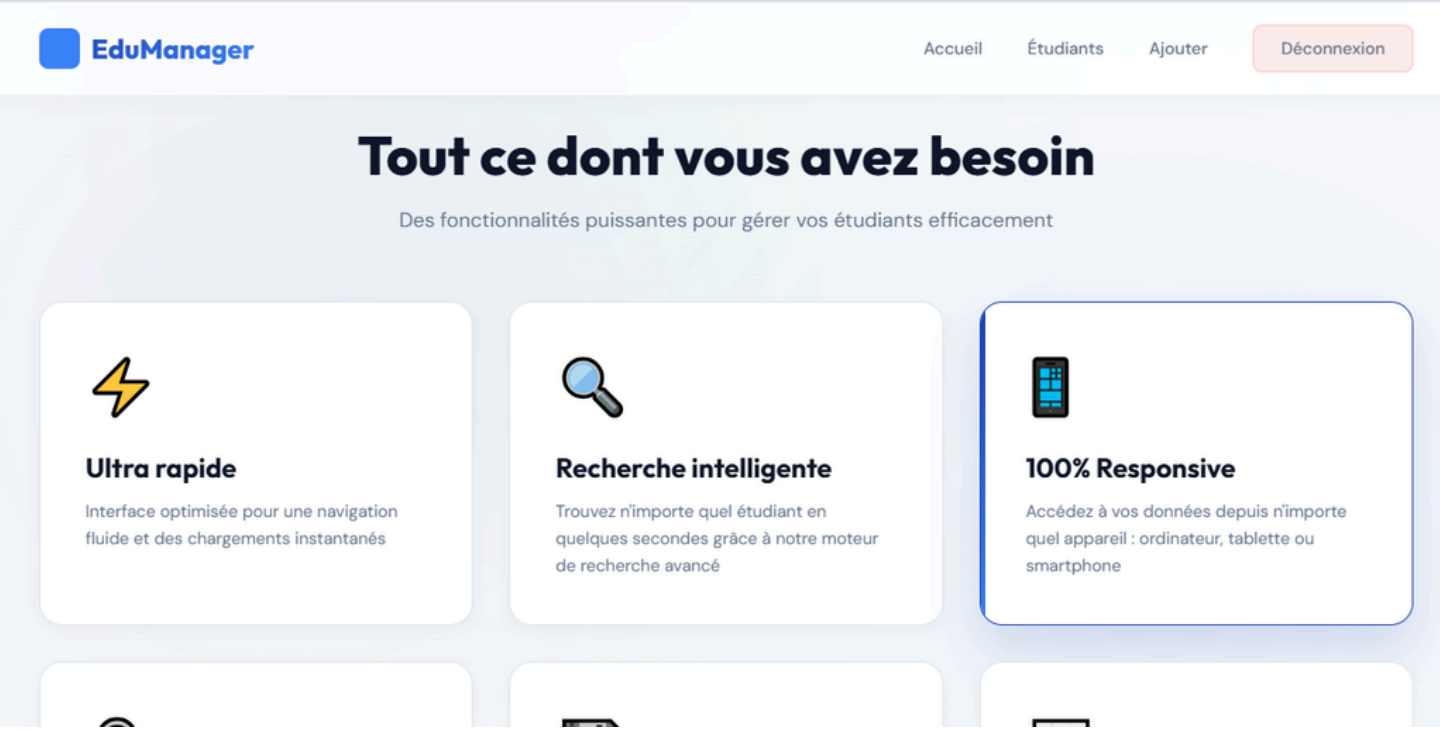
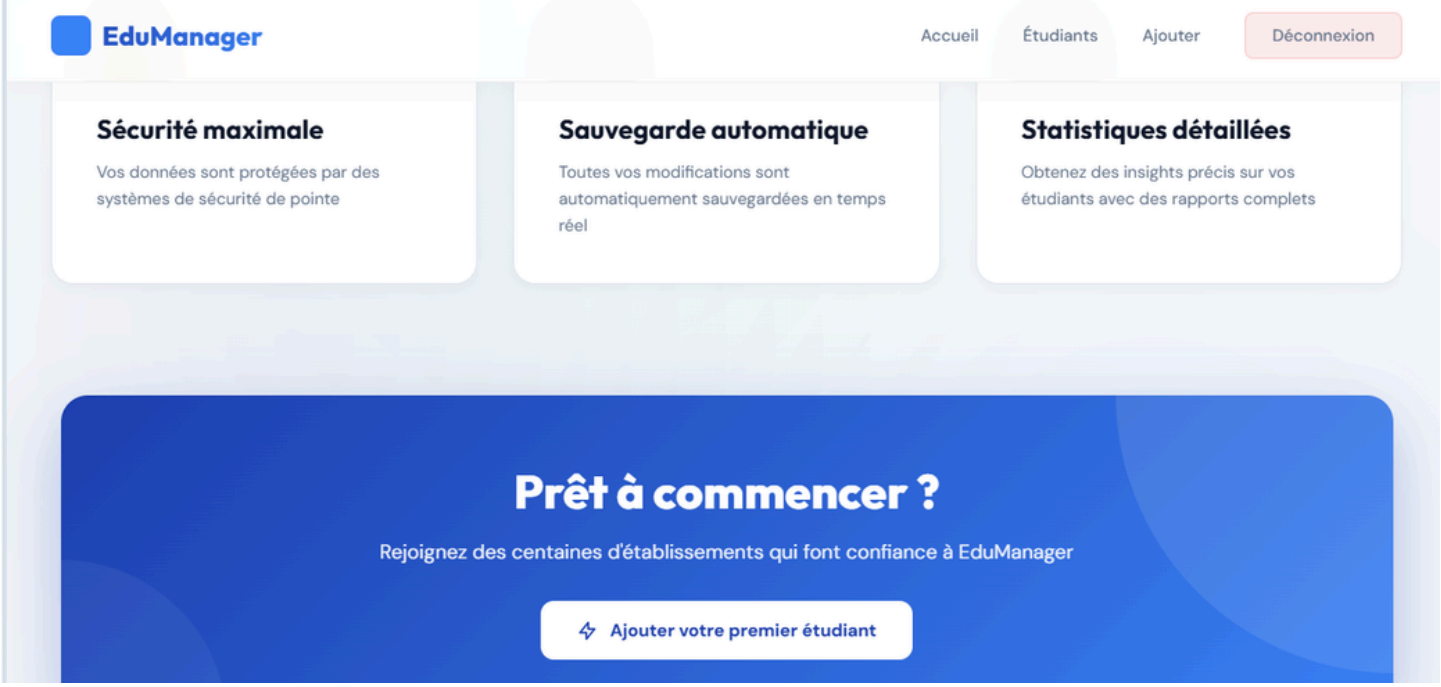
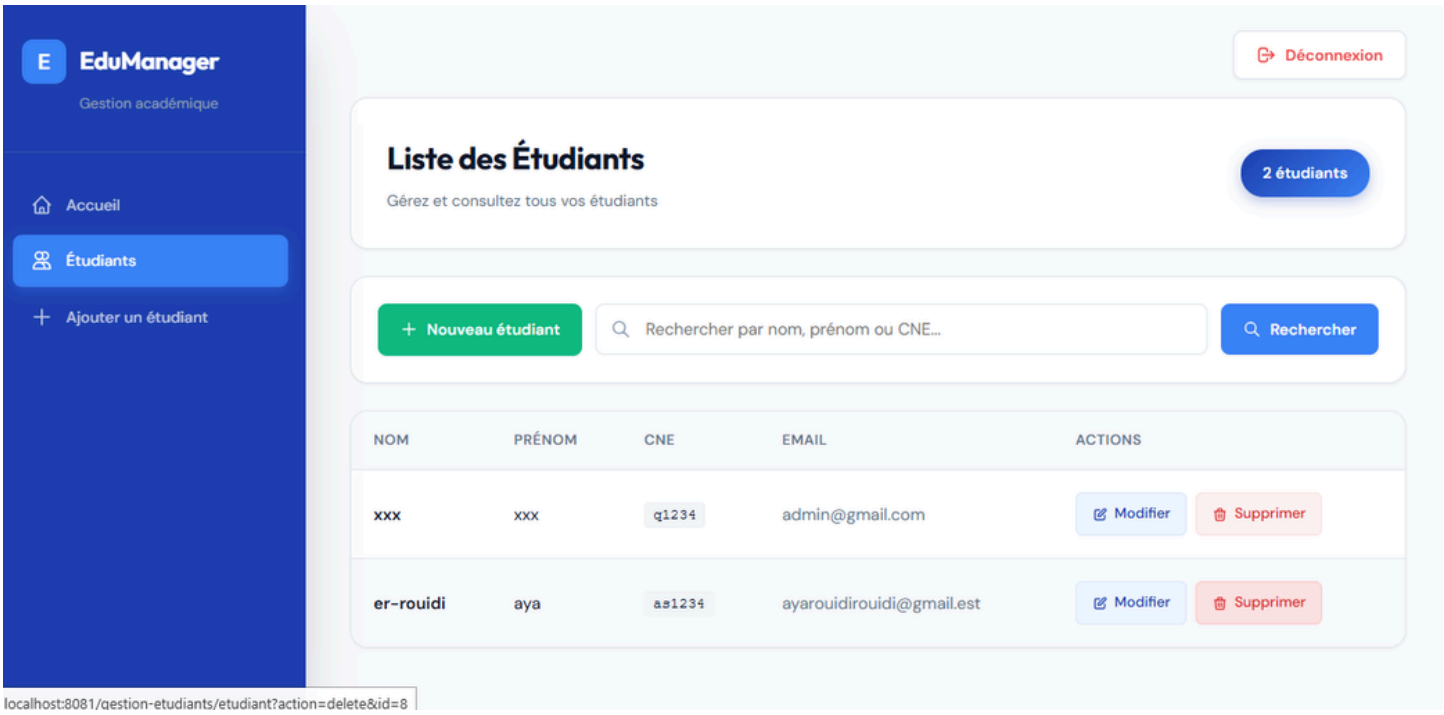


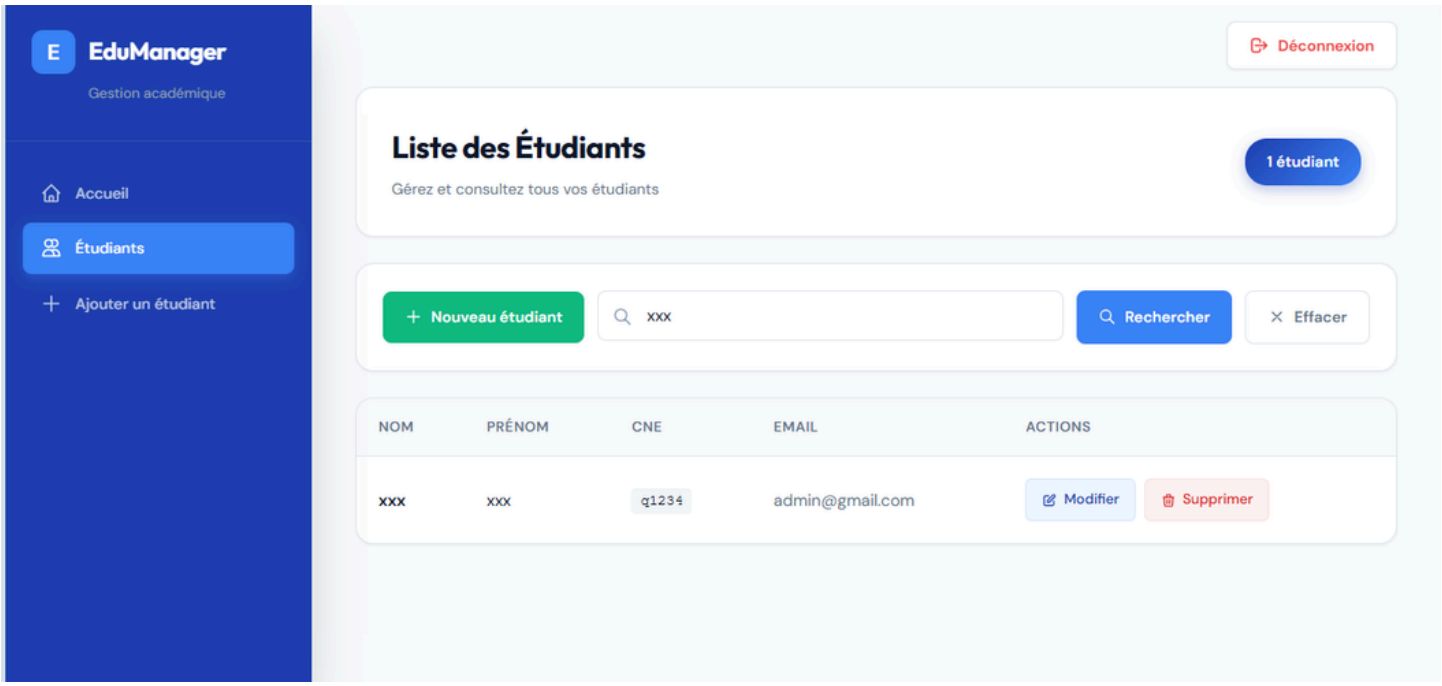
Image 3 — Section Avantages et Appel à l'action



10.2 Page Liste des Étudiants (avec recherche active)

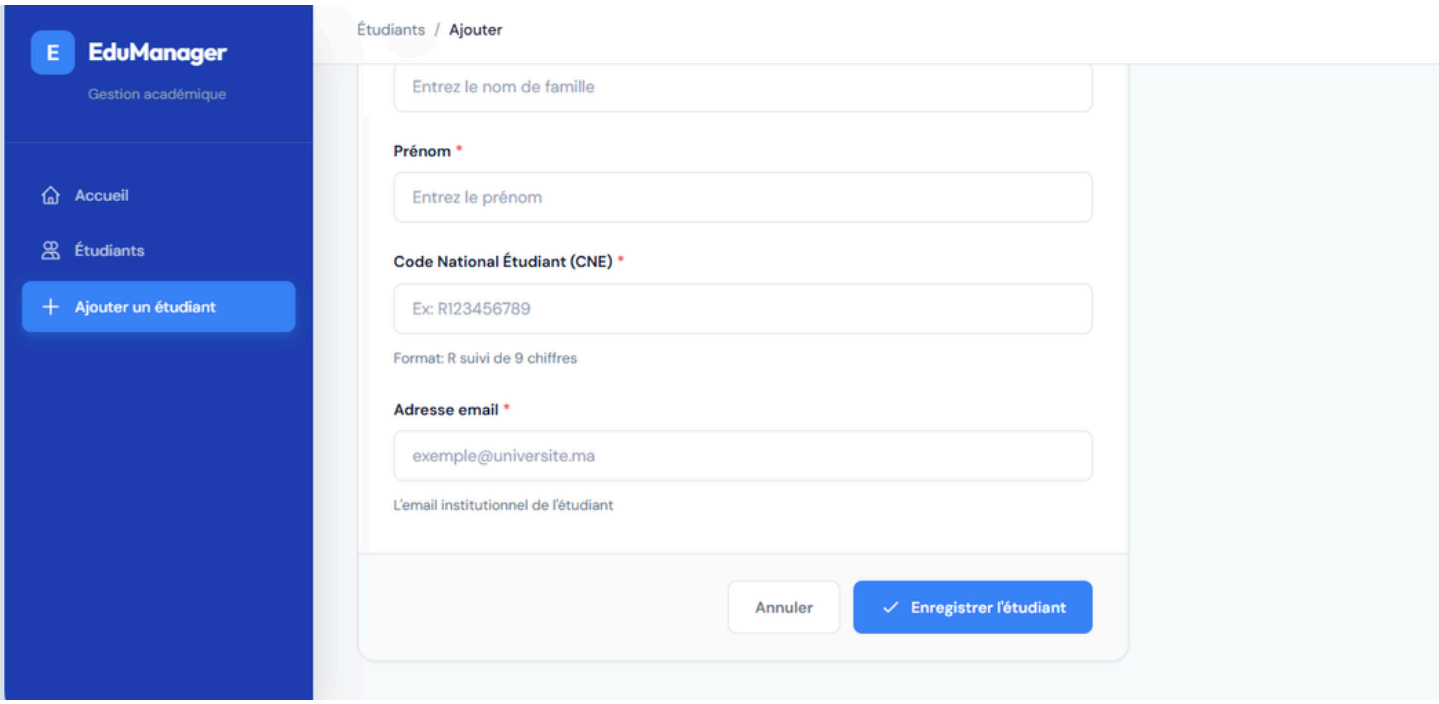
Description : Cette page affiche l'ensemble des étudiants enregistrés dans la base de données sous forme de tableau. Elle permet également d'effectuer des actions telles que la modification ou la suppression.





10.3 Page Ajouter un Étudiant

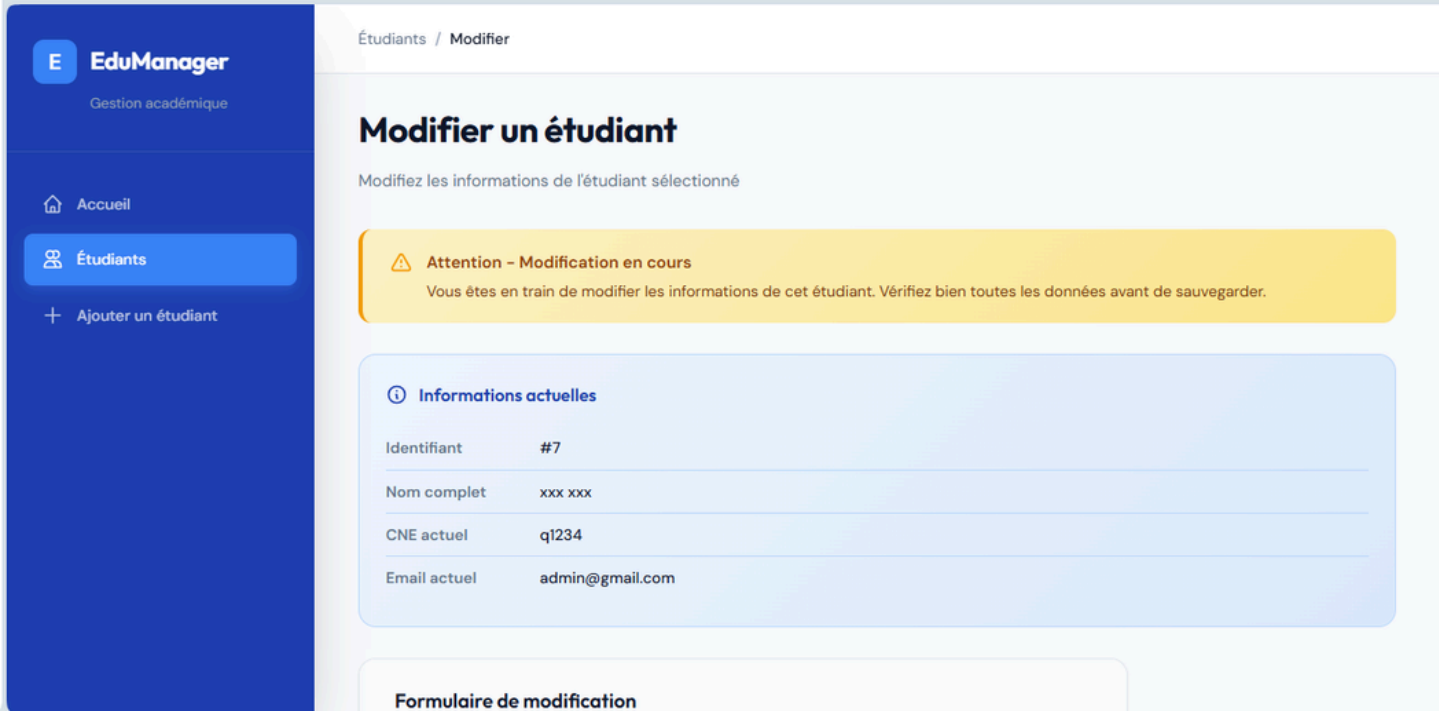
Description : Cette interface contient un formulaire permettant d'ajouter un nouvel étudiant. Les données saisies sont envoyées au serveur et enregistrées dans la base de données via la méthode doPost() du EtudiantServlet.

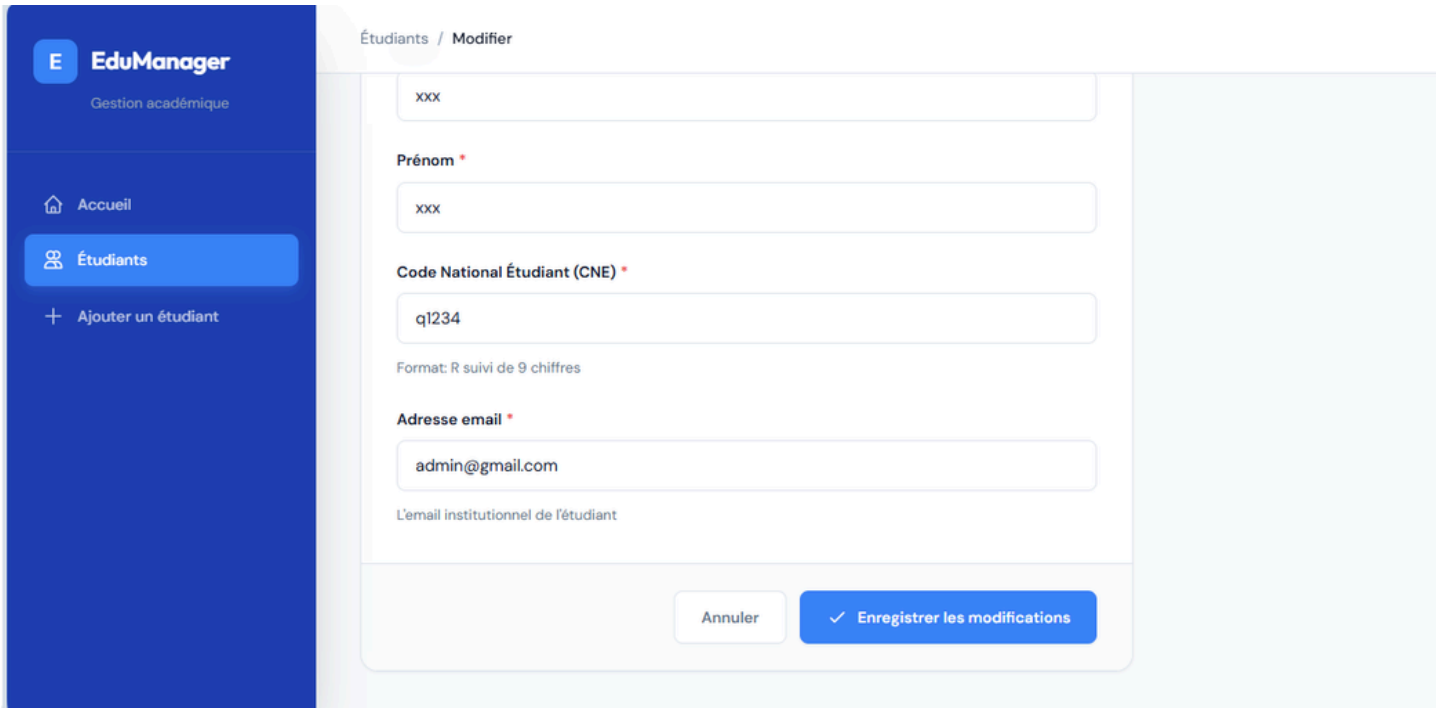


10.4 Page Modifier un Étudiant

Description : Cette page permet de modifier les informations d'un étudiant existant .Elle se compose de trois sections principales :

- Une **alerte orange** intitulée *« Attention – Modification en cours »* avertit l'utilisateur qu'il est en train de modifier les informations d'un étudiant et l'invite à vérifier toutes les données avant de sauvegarder.
- Un **encadré bleu clair** intitulé *« Informations actuelles »* affiche en lecture seule les données existantes de l'étudiant sélectionné : son identifiant (*#7*), son nom complet (*xxx xxx*), son CNE actuel (*q1234*) et son email actuel (*admin@gmail.com*).
- Un **Formulaire de modification** situé en dessous permet à l'administrateur de mettre à jour les informations de l'étudiant avant de les enregistrer.





10. Sécurité et gestion des sessions

- Utilisation de **HttpSession** pour gérer l'authentification.
- Protection des pages sensibles contre l'accès non autorisé.
- Séparation stricte entre l'espace administrateur et l'espace étudiant.

11.Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement web Java EE, notamment l'utilisation des Servlets, JSP et l'architecture MVC. L'application réalisée répond aux besoins de gestion des étudiants tout en offrant un espace sécurisé et simple d'utilisation pour les utilisateurs.

Ce travail constitue une base solide pouvant être enrichie et améliorée selon les besoins futurs.